

Mock Interview Feature

Implementation flow — denotes (denotes.co.in)

The mock interview feature lets a signed-in user upload their resume and be interviewed, one question at a time, by an AI interviewer grounded on that resume and a chosen role, skill set, difficulty, and time budget. The interview ends automatically — either when time runs out or the conversation reaches a turn ceiling — and produces a written evaluation with a 1–10 score. The whole feature sits behind a single feature flag and reuses the app's existing Gemini-primary / Groq-fallback LLM setup, so no new API keys were required.

Capabilities

- Resume upload and parsing (PDF via pdf-parse, DOCX via mammoth)
- Resume persistence per user — upload once, reuse on every later visit, replace anytime
- Conversational, one-question-at-a-time interview driven by an LLM (Gemini, falling back to Groq)
- Selectable interview duration (30 / 60 / 90 minutes) with dual server + client enforcement
- Typed or spoken answers (browser Speech Recognition, reusing the existing AudioRecorder component)
- End-of-interview feedback: summary, strengths, areas to improve, and a numeric score
- History of past interviews per user
- Feature-flagged (NEXT_PUBLIC_FF_INTERVIEW), toggleable at runtime from the admin dashboard

Architecture at a glance

```
resume file (PDF/DOCX, optional)      stored resume (if no new file)
    |                                  |
    v                                  v
app/api/interview/start/route.ts <-----'
    | (parses + saves resume if new; else loads saved copy)
    v
lib/interview/resumeParser.ts --> lib/interview/resumeStore.ts (MongoDB: userResumes)
    |
    v
lib/interview/generator.ts (Gemini --fallback--> Groq) --> first question
    |
    v
lib/interview/store.ts (MongoDB: interviewSessions) --> sessionId, question,
                                                         durationMinutes, startedAt
    |
    v
app/mock-interview/page.tsx (chat UI, countdown, voice/type toggle)
    | user answers
    v
app/interview-actions.ts: submitInterviewAnswer() --> next question OR final feedback
    |                                                         (whichever comes first: time
    v                                                         up, or turn-count ceiling)
lib/interview/store.ts (turns appended / session marked completed with feedback)
```

Two Next.js API surfaces are used deliberately: `app/api/interview/start` and `app/api/interview/resume` are plain route handlers (not server actions) because they move `multipart/form-data` file uploads; everything after a session exists — answering, ending early, listing history — goes through ordinary server actions in `app/interview-actions.ts`.

Feature flag and navigation wiring

The feature follows the app's existing feature-flag convention in `lib/featureFlags.ts`: a new `interview.enabled` flag (env default `NEXT_PUBLIC_FF_INTERVIEW`) and a `navigation.interview` flag for the nav link. Because the admin dashboard's flag panel walks the `featureFlags` object generically (`listToggleableFlags()`), both flags were automatically picked up there with no dashboard code changes — and, like every other flag in the app, the live value is actually read from MongoDB at runtime (via `lib/featureFlagOverrides.ts`), with the env var only acting as the factory default.

When the flag is on, a “Mock Interview” entry appears in the top navigation (`app/components/Navigation.tsx`) and as a quick-action tile on the authenticated home screen (`app/page.tsx`), both gated on `featureFlags.interview.enabled`.

Data model (MongoDB)

interviewSessions — one document per interview

Field	Type	Notes
<code>_id</code>	ObjectId	Session id, returned to the client as <code>sessionId</code>
<code>userId</code>	ObjectId	Owner; every read is scoped to this field
<code>resumeText</code>	string	Parsed plain-text resume used for this interview
<code>resumeFileName</code>	string	Original filename, shown in the history list
<code>skills</code>	string[]	Comma-separated input, split and trimmed
<code>role</code>	string	Target role, defaults to “Software Engineer”
<code>difficulty</code>	“easy” “medium” “hard”	Chosen on the setup screen
<code>durationMinutes</code>	30 60 90	Chosen on the setup screen
<code>status</code>	“in_progress” “completed”	Flips once feedback is generated
<code>turns</code>	InterviewTurn[]	{ question, answer? } per exchange
<code>feedback</code>	object (optional)	summary, strengths[], improvements[], score
<code>createdAt / updatedAt</code>	Date	<code>createdAt</code> anchors the duration deadline
<code>completedAt</code>	Date (optional)	Set when status becomes completed

userResumes — one document per user

Added for resume persistence: a single collection keyed by the user's own id (`_id == userId`), upserted whenever a new resume is uploaded. There is no history of past resumes — only the current one.

Field	Type	Notes
<code>_id</code>	ObjectId	Equal to the user's id (one document per user)
<code>resumeText</code>	string	Parsed plain-text resume
<code>resumeFileName</code>	string	Original filename, shown as “Resume on file”
<code>updatedAt</code>	Date	Shown to the user as the last-updated date

Flow 1 — Resume upload and persistence

Resumes used to be re-uploaded on every interview; they are now saved per user so a returning candidate is never asked twice.

- 1 On mount, `app/mock-interview/page.tsx` calls `GET /api/interview/resume`, which looks up the signed-in user's document in `userResumes` and returns its filename and last-updated time (or `{ resumeFileName: null }` if none exists yet).
- 2 If a resume is on file, the setup screen shows a compact “Resume on file” card instead of an upload field, with a **Replace** button that reveals the file input again.
- 3 If the user picks a new file (first time, or via Replace), it is validated client-side (`validateResumeFile`: rejects legacy .doc, non-PDF/DOCX, and files over 5MB) before anything is sent to the server.
- 4 On **Start interview**, the file — if a new one was chosen — is attached to the `multipart/form-data` POST to `/api/interview/start`; if the user kept their existing resume, no file is sent at all.
- 5 The route parses a newly uploaded file with `lib/interview/resumeParser.ts` (`dynamic import()` of `pdf-parse` or `mammoth`, text clamped to 20,000 characters) and immediately calls `saveUserResume()`, overwriting any previous resume for that user. When no file was sent, it instead calls `getUserResume()` and returns a 400 error if nothing is saved yet.

`lib/interview/resumeStore.ts` — upsert pattern

```
export async function saveUserResume(userId, input) {
  const collection = await getCollection();
  await collection.updateOne(
    { _id: new ObjectId(userId) },
    { $set: { resumeText: input.resumeText,
              resumeFileName: input.resumeFileName,
              updatedAt: new Date() } },
    { upsert: true }
  );
}
```

Flow 2 — Starting an interview

POST `/api/interview/start` (Node.js runtime, 60s max duration) does all the one-time setup work for a session:

- 1 Authenticate the request (`auth()`) and confirm at least one LLM provider is configured; otherwise return 503.
- 2 Read and validate the form fields: skills (split on commas), role (default “Software Engineer”), difficulty (fallback “medium”), duration (must be 30, 60, or 90; fallback 30).
- 3 Resolve the resume text — parse-and-save a new upload, or load the one already on file (Flow 1).
- 4 Call `generateInterviewTurn()` with an empty turn list to get the interviewer's opening question.
- 5 Persist a new document via `createInterviewSession()`: resume, skills, role, difficulty, duration, and the first question as `turns[0]`.

- 6 Respond with `{ sessionId, question, durationMinutes, startedAt, resumeFileName }`. The client uses `startedAt + durationMinutes` to seed its own countdown, rather than guessing a deadline independently.

Flow 3 — Conducting the interview

`lib/interview/generator.ts` is a self-contained LLM wrapper, structurally parallel to the existing RAG chat generator but with a different prompt/response contract: it asks the model for strict JSON, one of two shapes.

```
{ "type": "question", "question": "..."} 
```

```
{ "type": "feedback", "summary": "...", "strengths": [...],  
  "improvements": [...], "score": 7} 
```

The system prompt instructs the model to act as an experienced technical interviewer, basing questions on the resume and chosen skills, mixing behavioral and technical questions at the chosen difficulty, and asking natural follow-ups. `parseResult()` strips stray markdown fences some models add despite instructions, and falls back to treating the raw text as a question if JSON parsing fails outright — a malformed response degrades gracefully instead of breaking the interview.

Generation tries Gemini first (`generative-language.googleapis.com`, direct fetch call) and falls back to Groq (OpenAI-compatible client) on a quota/rate-limit error, missing key, or any other failure — mirroring the existing RAG chat feature's provider setup so no new credentials were needed.

Per-turn flow (after the first question)

- 1 The user types an answer, or records one via the **Speak** toggle, which shows the existing `AudioRecorder` component and transcribes with the browser's native Speech Recognition API — entirely client-side, no new backend transcription call, and distinct from the Whisper-based pipeline Audio Notes uses. The transcript lands in the editable textarea rather than auto-submitting, so a misheard word can be fixed first.
- 2 Submitting calls the `submitInterviewAnswer(sessionId, answer)` server action, which loads the session, records the answer against the last turn, and decides whether this is the final turn (Flow 4).
- 3 `generateInterviewTurn()` is called again with the full transcript. A “question” result is appended to turns and shown in the chat; a “feedback” result completes the session (Flow 5).

Flow 4 — Duration selection and enforcement

The candidate picks 30, 60, or 90 minutes on the setup screen. The interview wraps up automatically — triggering final feedback — whichever of two independent checks fires first:

- **Time runs out.** `isInterviewTimeUp()` in `lib/interview/store.ts` compares `createdAt + durationMinutes` against the current time. It is checked server-side on every answer submission, so it is enforced even if the client's own countdown is closed or tampered with.
- **Turn-count ceiling.** `INTERVIEW_MAX_TURNS` (default 40, raised from an earlier default of 6 once duration became the primary stop signal) is a defensive backstop only — high enough that a normal 30/60/90-minute interview never reaches it.

```
// app/interview-actions.ts
const isFinal = turns.length >= interviewConfig.maxTurns || isInterviewTimeUp(doc);
```

On the client, the interview header shows a live countdown (`formatCountdown()`), ticking every second from `startedAt + durationMinutes`), turning red in the final minute, and calls the same “end early” handler the moment it reaches zero — purely for responsiveness, since the server enforces the real deadline independently.

Flow 5 — Ending the interview and feedback

An interview can end two ways, both converging on the same feedback shape (`{ summary, strengths[], improvements[], score }`):

- **Automatically**, inside `submitInterviewAnswer()`, when `isFinal` is true after the candidate's latest answer — the model is asked for feedback directly instead of another question.
- **Manually**, via the **End interview** button, calling `endInterviewEarly()`. This drops any trailing unanswered question so the model is evaluated only on what was actually answered, and defends against a non-feedback model response with a neutral fallback (“Interview ended early.”, score 0) rather than failing.

Either path calls `completeInterviewSession()`, which sets `status: “completed”`, stores the feedback object, and stamps `completedAt`. The UI then renders the summary, a bulleted strengths list, a bulleted improvements list, and the 1–10 score.

Flow 6 — Past interviews

`getMyInterviewSessions()` lists up to 20 of the signed-in user's most recent sessions (role, difficulty, duration, status, score, date), shown under the setup form so a candidate can see their history and progress before starting another attempt.

File reference

File	Role
lib/interview/config.ts	Reuses RAG's Gemini/Groq keys; INTERVIEW_MAX_TURNS and max resume size
lib/interview/resumeParser.ts	PDF/DOCX -> plain text (pdf-parse / mammoth), 20k-char clamp
lib/interview/resumeStore.ts	Per-user resume persistence (userResumes collection)
lib/interview/generator.ts	Gemini-primary / Groq-fallback question + feedback generation
lib/interview/store.ts	interviewSessions CRUD, duration deadline check
app/api/interview/start/route.ts	Starts a session: resolves resume, gets first question, persists
app/api/interview/resume/route.ts	Returns the current user's saved resume metadata
app/interview-actions.ts	submitInterviewAnswer, endInterviewEarly, getMyInterviewSessions
app/mock-interview/page.tsx	Setup form, chat UI, countdown, voice/type toggle, feedback screen
lib/featureFlags.ts	interview.enabled / navigation.interview flags
app/components/Navigation.tsx	Nav link, gated on the feature flag
app/page.tsx	Home-screen quick-action tile, gated on the feature flag
types/resume-parsers.d.ts	Ambient types for pdf-parse / mammoth (neither ships its own)
next.config.js	serverExternalPackages: pdf-parse, mammoth
MOCK_INTERVIEW_SETUP.md	Setup, env vars, and feature documentation

Configuration

Field	Type	Notes
NEXT_PUBLIC_FF_INTERVIEW	flag	Turns the whole feature on; no new LLM keys needed if RAG chat is already
INTERVIEW_MAX_TURNS	number (default 40)	Defensive turn-count ceiling, independent of duration
INTERVIEW_MAX_RESUME_BYTES	number (default 5MB)	Upload size limit, checked client- and server-side